

Low-Level Design (LLD)

Technical Specifications & Logic for RAG-HITL Assistant

1. State Management & Data Structures

The system's state is managed through a centralized `AgentState` object within `LangGraph`. This ensures that information like the user's query and the retrieved document context is preserved across nodes.

Field	Type	Description
query	str	The original user input.
context	str	Combined text chunks from the vector database.
answer	str	Final response generated by the system.
needs_escalation	bool	Boolean flag indicating if a human agent is required.

2. Node-Level Logic Specifications

2.1 Retrieval Logic (retrieve_node)

This node interfaces with the **ChromaDB** vector store. It utilizes `similarity_search` with a parameter of `k=3` to fetch the top three most relevant document chunks based on the cosine similarity of embeddings.

2.2 Decision Routing (generate_node)

This node contains the core conditional logic. It evaluates the quality of the retrieved context. If the context length is below a specific threshold (e.g., 100 characters), it concludes that the knowledge base does not contain the answer and sets `needs_escalation = True`.

2.3 Human-in-the-Loop (escalate_node)

Triggered only when `needs_escalation` is `True`. This node halts automated execution and provides a `manual_input()` interface for a human operator to provide the correct response, which is then saved to the `answer` state.

3. Data Processing Hyperparameters

Parameter	Value	Justification
Chunk Size	1000	Optimal for Llama-3 context window and semantic density.
Chunk Overlap	200	Maintains sentence continuity across splits.
Embedding Model	all-MiniLM-L6-v2	Fast, local, and accurate 384-dimensional vectors.
LLM Model	llama3-8b-8192	Chosen for low latency and high reasoning capability via Groq.